

Efficient Rectification Signal Validation for Optimal Functional ECO Patch Generation

Tzu-Yu Tung^{*†}, Yu-Ling Hsu^{*‡}, Shao-Lun Huang[‡] and Chung-Yang Ric Huang^{*†}

[†]National Taiwan University, [‡]Cadence Design Systems

tzuyutung0403@gmail.com, {yulingh, shaolun}@cadence.com, cyhuang@ntu.edu.tw

Abstract—Synthesis-based functional Engineering Change Order (ECO) algorithms, as classified in [1], are particularly effective for addressing functional bugs. These algorithms typically involve two primary steps: (1) identifying rectification signals to address functional mismatches, and (2) generating patch circuits based on these signals. While much of the existing research focuses on enhancing step (2), step (1) often remains somewhat ad hoc and inefficient.

In this paper, we propose a novel approach for systematically collecting and validating all possible sets of rectification signals from a given set of candidates. Leveraging a heuristic for grouping and ranking rectification signals, our ECO flow efficiently identifies minimal patches while achieving a highly competitive runtime. Our contributions include three key innovations: a heuristic for identifying high-quality rectification candidates, an efficient algorithm for validating all feasible sets of rectification signals, and a signal grouping and ranking technique that ensures minimal patch size. When integrated with an open-source patch generation tool, our method demonstrates an average reduction of 44% in patch sizes compared to a leading commercial ECO tool on benchmark circuits.

Keywords—ECO, rectification point, quantifier elimination, SAT, QBF

I. INTRODUCTION

In modern VLSI design practices, functional Engineering Change Order (ECO) plays a vital role in addressing design errors and accommodating specification changes. As outlined in [1], ECO algorithms can be broadly classified into three categories: fault-model-based, sweeping-based, and synthesis-based.

For modifications involving common design errors—such as gate-type changes, missing or extra inverters, or rewiring—fault-model-based ECO algorithms [2]–[4] are the most effective, achieving minimal patch costs as noted in [1]. For more complex modifications that extend beyond fault models but remain confined to specific modules or sub-circuits, sweeping-based ECO algorithms [5]–[7] are more efficient. These algorithms minimize patches by merging functionally equivalent portions between the original and corrected (golden) circuits.

However, when modifications affect the design globally—such as altering the logical expression in a "case" statement, thereby impacting the satisfiability and observability don't-cares—rectifications are no longer local. In such scenarios, neither fault-model-based nor sweeping-based algorithms are effective. Synthesis-based ECO algorithms [8]–[12], by contrast, excel in addressing such functional changes [1]. These algorithms employ error-diagnosis techniques to identify potential rectification signals and then use synthesis techniques to generate patches that resolve the functional discrepancies.

Synthesis-based ECO algorithms can be formalized as follows. Let $F(\mathbf{x})$ and $G(\mathbf{x})$ represent the functions of the old

and golden multiple-output circuits, respectively, where $\mathbf{x}$ is the set of primary inputs (PIs). Let $\mathbf{r}$ denote a set of internal signals in F . The function $F(\mathbf{x})$ is multi-rectifiable at signals $\mathbf{r}$ with respect to $G(\mathbf{x})$ if and only if:

$$\forall \mathbf{x}. \exists \mathbf{r}. F(\mathbf{x}, \mathbf{r}) \equiv G(\mathbf{x}) \quad (1) [12]$$

The above single alternation $\forall\exists$ -formula is a 2QBF (Quantified Boolean Formula) problem. Solving this specific 2QBF problem is classified as a Π_2^P -complete problem [12] [13]. Earlier researches employed symbolic techniques such as Binary Decision Diagrams (BDDs) and truth tables to fix erroneous designs [14][15]. However, these approaches often suffered from runtime or memory explosion as design complexity increased.

To address this inefficiency, the authors in [9] proposed a SAT-based technique that assumes errors can be rectified on a single signal. This method computes patches through functional interpolation and leverages the scalability of modern SAT solvers. While this single-signal rectification assumption is generally valid—since rectifications can often revert to fixing one primary output (PO) at a time—it is not inherently optimal.

The authors in [11] then enhanced the above algorithm by incrementally generating partial patches on the rectification signals. This work naturally extended the SAT-based functional ECO technique to multi-error rectifications. Nevertheless, the order of the incremental fixes is somewhat arbitrary, and a poorly chosen sequence may lead to non-convergence on the patch generation. To address this limitation, the same authors later devised a multi-patch generation approach by interpolation with cofactor reduction [12]. Given a set of rectification signals, this new algorithm could fix multi-errors on the circuit simultaneously.

While these synthesis-based functional ECO algorithms appear to provide optimal patch solutions, their optimality is constrained by the pre-determined set of rectification signals. Since rectification signals are typically identified through error-diagnosis techniques, they often address only the error-triggering input vectors rather than the full scope of differences between the old and golden circuits. Consequently, these ECO algorithms frequently need to enumerate various combinations of rectification candidates and settle on the first set of signals that satisfies Equation (1).

Note that for clarification, we call the signals suggested by the error-diagnosis techniques the "rectification candidates", denoted as $\mathbf{r}_F$, and any of its subsets that satisfies Equation (1) the "rectification signals", denoted as $\mathbf{r}$. Therefore, to find the optimal ECO patch, we should enumerate all the possible $\mathbf{r}$ in $\mathbf{r}_F$ and compute:

$$\min_{\mathbf{r}: \mathbf{r} \subseteq \mathbf{r}_F} \text{cost}(\mathbf{r}), \text{ in which } \forall \mathbf{x}. \exists \mathbf{r}. F(\mathbf{x}, \mathbf{r}) \equiv G(\mathbf{x}) \quad (2)$$

where $\text{cost}()$ is the cost function for the optimal patch based on $\mathbf{r}$.

However, given that generating an optimal patch is already computationally expensive, it seems impractical to

iterate through all the possible subsets $\mathbf{r}$'s to further optimize the patch.

To cope with this challenge, in this paper, we propose an novel rectification signal validation technique so that we can identify all the possible sets of rectification signals very efficiently. This is achieved by checking an approximated function, instead of Equation (1), for the rectifiability of a given set of candidate signals. The approximated function is constructed based on quantifier elimination and functional unateness techniques.

In addition, we further devise several heuristic algorithms so that we can obtain the optimal patch circuit with the fewest possible number of calls to the expensive patch generation routine. The experimental results show that our rectification signal validation and collection algorithm, when combined with an open-source ECO engine, can outperform a commercial ECO tool by an average of 44% in patch sizes on benchmark circuits.

The rest of the paper is organized as follows. In Section II, we first introduce our three-stage function ECO flow. The most important technique, the rectification signal validation algorithm, is presented in Section III. Section IV demonstrates the experimental results and Section V concludes the paper.

II. OUR FUNCTIONAL ECO FLOW

With the efficient rectification signal validation technique, our functional ECO algorithm can be outlined in the following three stages: (1) Preparation Stage, where we select the rectification candidates based on error diagnosis techniques [14][15]; (2) Validation Stage, where we apply an approximated function to collect all the sets of rectification signals; and (3) Patch Generation Stage, where we rank the sets of rectification signals and obtain the optimal patch with very few calls to the patch generation tool.

A. Preparation Stage

The choice of the rectification signals greatly determines the final quality of the patch. Intuitively speaking, if we can select a candidate closer to the bug location, it is more likely that the patch to fix the difference will be smaller. Therefore, we propose a *SuspiciousScore* function to evaluate the likelihood of a signal to be the bug location in the circuit.

The fundamental concept of this approach is to quantify a signal's ability to correct counterexamples. As illustrated in the example of Figure 1, the circuit is presented in an And-Inverter-Graph (AIG) format where each circle is an AND gate and dot and solid lines correspond to inverted and non-inverted fanins, respectively. With the counterexample "1101", flipping the value of gate(9) is able to fix the bug while flipping that of gate(7) will not. Therefore, only gate(9)'s suspicious score will be increased in this case.

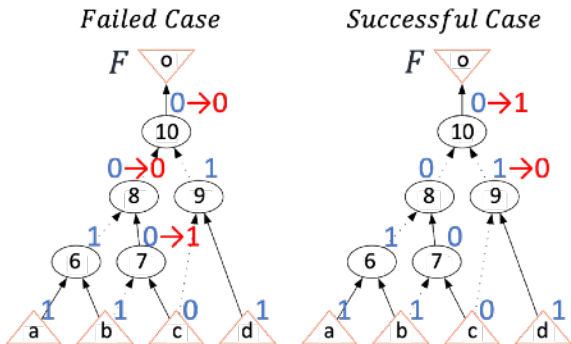


Fig. 1. Correcting a counterexample by flipping a signal's simulation value

In this preparation stage, we first perform random simulations and collect the set of counterexamples $\mathbf{cex}$ if there is a discrepancy on any output of the old (F) and golden (G) circuits. For pairs of internal signals that have the same responses on all the simulation patterns, we use the SAT solver to prove their equivalence. If asserted, they will be merged and form a new input frontier [16]. Otherwise, the patterns to distinguish the pairs will be tested if they are indeed counterexamples. In addition, we manage to somewhat balance the numbers of counterexamples to witness the non-equivalence among circuit outputs. For those outputs with relatively few counterexamples, we will resort to the SAT solver to generate more counterexamples.

Once we have collected the counterexamples, we will compute the *SuspiciousScore* of a signal s based on three factors:

- $N_{\mathbf{cex}}$: Number of counterexamples it can fix when flipping its simulation value.
- S_{cex} : For each counterexample cex in $\mathbf{cex}$, the number of signals that can rectify this counterexample. This indicates the rarity of this signal's capability in fixing this counterexample.
- D_{max} : The longest distance from this signal to the circuit outputs. By introducing this factor, we favor rectification signals that are closer to circuit inputs.

Given a signal s , its *SuspiciousScore* is calculated as:

$$SuspiciousScore(s) = \left(\sum_{N_{cex}} \frac{1}{S_{cex}} \right) \cdot (1 + D_{max}) \quad (3)$$

Note that we only need to compute the *SuspiciousScore*'s for the signals in F , and as we calculate them in the topological order, the time complexity is $O(S \cdot N)$, where S is the number of signals in F and N is the total number of counterexamples.

Lastly, we select the rectification candidates with the top n *SuspiciousScore*, where n is a programmable variable so that the selected candidates should cover all the non-equivalent POs.

B. Validation Stage

As indicated in Equation (2), there are two runtime bottlenecks in computing the optimal patch circuit: The first is to solve the 2QBF equation for every set of rectification signals, and the second is to call the patch generation process many times to seek the minimal patch.

To alleviate the second issue, we create this "validation stage" to collect all the possible sets of rectification signals first, leaving the expensive patch generation to the next stage. This enables the grouping and ranking on the sets of rectification signals, and if we can ensure that the first few sets of rectification signals can indeed render the optimal patch, we can early terminate the search process and largely reduce the number of calls to the patch generation routine.

For the runtime issue in solving the enormous amount of 2QBF problems, we devise an efficient rectification signal validation algorithm as illustrated in Algorithm I below. Note that instead of solving the 2QBF problem in Equation (1), we first invert it to:

$$\exists \mathbf{x} . \forall \mathbf{r} . F(\mathbf{x}, \mathbf{r}) \neq G(\mathbf{x}) \quad (4)$$

and then construct an approximated quantifier-free function $H(\mathbf{x}, s)$ to remodel $\forall \mathbf{r} . F(\mathbf{x}, \mathbf{r}) \neq G(\mathbf{x})$ (details will be presented in the next section). Therefore, the 2QBF can be reduced to 1QBF (i.e. $\exists \mathbf{x} . H(\mathbf{x}, s)$) and be solved by SAT.

In Algorithm I, we first introduce a set of auxiliary selection variables $\mathbf{s}_F$ (Line 4) in order to enumerate all the

Algorithm I. Rectification Signal Validation

1. Function *RectificationSignalValidation*($\mathbf{x}, \mathbf{r}_F$):
 2. // $\mathbf{x}$: set of circuit inputs
 3. // $\mathbf{r}_F$: set of rectification candidates
 4. Define $\mathbf{s}_F$: set of selection variables // Note: $|\mathbf{s}_F| = |\mathbf{r}_F|$
 5. Define $H(\mathbf{x}, s)$: an approximated function with respect to a selection variable s in $\mathbf{s}_F$
 6. Let $F'(\mathbf{x}) = F(\mathbf{x})$ // to be remodeled by $H(\mathbf{x}, s)$
 7. for each signal r_i in $\mathbf{r}_F$ and s_i in $\mathbf{s}_F$
 8. $H(\mathbf{x}, s_i) \leftarrow \text{ConstructProofModel}(\mathbf{x}, r_i)$
 9. endfor
 10. // The proof model for $\mathbf{s}_F$ is denoted as $H(\mathbf{x}, \mathbf{s}_F)$
 11. Let $S = \bigvee_{s_i \in \mathbf{s}_F} s_i$ // to characterize the search on $\mathbf{s}_F$
 12. Let $R = \emptyset$ // to record the sets of rectification signals
 13. while $((\vec{\mathbf{s}}_F \leftarrow \text{SAT}_1(S)) = \text{SATISFIABLE})$
 14. $\vec{\mathbf{s}}_F$ is an assignment on variables in $\mathbf{s}_F$
 15. $H'(\mathbf{x}, \mathbf{s}_F) \leftarrow \text{ApplyValue}(\vec{\mathbf{s}}_F, H(\mathbf{x}, \mathbf{s}_F))$
 16. If $((\vec{\mathbf{x}} \leftarrow \text{SAT}_2(H'(\mathbf{x}, \mathbf{s}_F))) = \text{UNSATISFIABLE})$
 17. $\vec{\mathbf{x}}$ is an assignment on variables in $\mathbf{x}$
 18. $\mathbf{r} \leftarrow \text{UnsatAnalysis}(\mathbf{r}_F, \text{SAT}_2)$
 19. $R \leftarrow \text{AddRecSignalSet}(\mathbf{r}, R)$
 20. $\text{SAT}_1 \leftarrow \text{AddBlockingCls}(\mathbf{r}, \mathbf{s}_F, \text{SAT}_1)$
 21. else
 22. $\vec{\mathbf{r}} \leftarrow \text{SatGen}(\mathbf{r}_F, \text{SAT}_2)$
 23. $\text{SAT}_1 \leftarrow \text{AddBlockingCls}(\vec{\mathbf{r}}, \vec{\mathbf{s}}_F, \text{SAT}_1)$
 24. endif
 25. endwhile
 26. Return R
-

combinations of the rectification candidates. Each selection variable s_i in $\mathbf{s}_F$ is to determine whether the corresponding rectification candidate r_i is selected into the set of rectification signals. If yes, then the circuit $F(\mathbf{x})$ is remodeled with the approximated function $H(\mathbf{x}, s_i)$ (Lines 5-10). Otherwise, $F(\mathbf{x})$ remain the same with respect to r_i .

To collect all the possible sets of rectification signals, we employ two satisfiability solvers SAT_1 and SAT_2 . SAT_1 is to enumerate the assignments on the selection variables $\mathbf{s}_F$. Initially, we define a Boolean function S as the disjunction of all the selection variables s_i 's (Line 11). Obviously, it is satisfiable. Note that the decision variable order in SAT_1 follows the rank of the suspicious scores of variables in $\mathbf{s}_F$ so that the one with higher score will be picked first. Once we get an assignment $\vec{\mathbf{s}}_F$, we apply it to simplify the remodeled circuit $H(\mathbf{x}, s)$ and call SAT_2 to check the satisfiability (Lines 13 - 16).

If SAT_2 returns UNSATISFIABLE, we find a set of potential¹ rectification signals $\mathbf{r}$ and we call the UNSAT

analysis routine to minimize the number of necessary variables in $\mathbf{r}$. This can reduce the set of rectification signals and thus minimize the patch (Lines 16-18). We then record $\mathbf{r}$ in R and add a blocking clause of $\mathbf{s}_F$ based on $\mathbf{r}$ to SAT_1 .

Otherwise, if SAT_2 returns SATISFIABLE, it implies that the current assignment cannot lead to a valid rectification. Therefore, we preform the SAT generalization to find the minimal assignment of $\vec{\mathbf{r}}$ and add the corresponding blocking clause by $\vec{\mathbf{s}}_F$ to SAT_1 (Lines 22-23).

The above procedure (Lines 13-25) will repeat until all the combinations of the rectification candidates $\mathbf{r}_F$ have been tested.

C. Patch Generation Stage

Once we have collected all the sets of potential rectification signals, we can call the patch generation tool for each set of signals and obtain the optimal patch circuit. However, considering the large number of the sets and the long runtime of the patch generation routine, we should minimize the number of patch generation calls and hopefully the optimized patch can be found in the first few calls.

We observed that many sets of the rectification signals are quite similar and empirically it shows that their patch costs are almost the same. Therefore, we group the sets of the rectification signals by their similarities and pick the one that is firstly obtained in Algorithm I as the representative. The grouping criteria we adopt here is simple: if there are more than 50% of common signals among two sets, we will place them in the same group.

We also noticed a trend that the closer the rectification signals are to the PIs, the smaller the patch circuit it may be. Therefore, for each rectification signal, we define its "level" as the largest distance from the PIs to this signal and then sort the sets by the summation of the levels of the rectification signals in the set. Note that by picking the sets with the smaller summations of signal levels, we not only select the ones which are closer to PIs, but also favor the sets with smaller numbers of rectification signals.

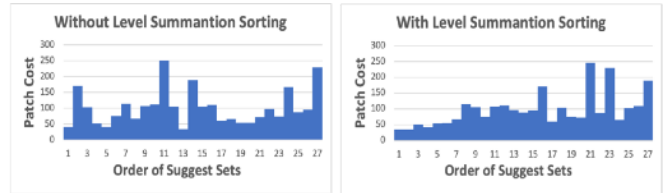


Fig. 2. Patch costs for different representative sets of rectification signals. Left figure are in the original order, and right one is sorted by the summation of the levels of the rectification signals.

As we generate the patches from the ordered sets, the next question is whether we have found the minimum patch and when we should stop the search. To answer this question, we implement a simulation annealing procedure where the search will be discontinued once there are accumulatively a few unsuccessful attempts to improve the patch cost. As demonstrated in the experimental results, our ECO flow can indeed obtain the minimal patches within just a few calls.

In addition, we also implement a simple heuristic to switch the patch generation from all-POs-at-a-time, group-by-group-POs, and one-PO-at-a-time. This is particularly helpful when the ECO problem involve multiple errors.

¹ We state it "potential" here because we remodel the circuit $F(\mathbf{x})$ with an approximated function $H(\mathbf{x}, s)$. It is likely that the suggested rectification signals may not render any valid patch. However, according to our experimental results, this rarely happens. We will cover the details in later sections.

III. MODELING AND IMPLEMENTATION OF THE RECTIFICATION SIGNAL VALIDATION ALGORITHM

As mentioned in the previous section, given a set of rectification candidates $\mathbf{r}_F$, we extract the sets of valid rectification signals by constructing an approximated quantifier-free function $H(\mathbf{x}, s)$ for the formula $\forall \mathbf{r}. F(\mathbf{x}, \mathbf{r}) \neq G(\mathbf{x})$ and using SAT solver to collect the sets of rectification signals from its unsatisfiability cores. We will explain in details about the modeling and implementation of the algorithm.

A. Quantifier Elimination for a Rectification Candidate

Given an universally quantified formula $\forall r. F(\mathbf{x}, r)$, we can transform it into a quantifier-free formula as the conjunction of its cofactors on the quantifier variable r : $F(\mathbf{x}, r=0) \wedge F(\mathbf{x}, r=1)$. However, if there are multiple quantifier variables $\mathbf{r}$, there will be exponential terms in the conjunctive formula with respect to the number of variables in $\mathbf{r}$.

Consider the case that r has only one fanout $u(r)$ which is unate to F . We can then describe F as $A \cdot u(r)$, where A is certain function that is independent of r . Therefore, the universal quantification of F on r can be rewritten as:

$$\forall r. F = \forall r. (A \cdot u(r)) = A \cdot (\forall r. u(r)) \quad (5)$$

Note that if the fanout function $u(r)$ is not too complicated, the elimination of the single quantifier variable r is relatively simple. For example:

$$\forall r. \text{AND}(r, a) = 0$$

$$\forall r. \text{OR}(r, a) = a$$

$$\forall r. (ra + \bar{r}b + c) = (a + c)(b + c)$$

For the case that r has multiple fanouts which are unate to F , F can be represented as $A \cdot u_1(r)u_2(r) \dots u_n(r)$ and the universal quantification can be rewritten as:

$$\forall r. F = A \cdot (\forall r. u_1(r))(\forall r. u_2(r)) \dots (\forall r. u_n(r)) \quad (6)$$

B. Constructing the Approximated Function $H(\mathbf{x}, \mathbf{s}_F)$

Before we introduce our algorithm for constructing the approximated function $H(\mathbf{x}, s)$ to remodel Equation (4), we should first answer the following questions:

- How to apply quantifier elimination to the miter circuit $F(\mathbf{x}, r) \neq G(\mathbf{x})$

The miter function $F(\mathbf{x}, r) \neq G(\mathbf{x})$ can be rewritten as $F(\mathbf{x}, r) \vee G(\mathbf{x}) \wedge (\bar{F}(\mathbf{x}, r) \vee \bar{G}(\mathbf{x}))$. As universal quantifier $\forall r$ can be distributed over the conjunctive operator ' $\wedge$ ' and $G(\mathbf{x})$ is independent of the signals r in F , the universal quantification over the miter function can be computed by:

$$\begin{aligned} \forall r. ((F(\mathbf{x}, r) \vee G(\mathbf{x})) \wedge (\bar{F}(\mathbf{x}, r) \vee \bar{G}(\mathbf{x}))) \\ = (\forall r. F(\mathbf{x}, r)) \vee G(\mathbf{x}) \wedge (\forall r. (\bar{F}(\mathbf{x}, r) \vee \bar{G}(\mathbf{x}))) \quad (7) \end{aligned}$$

- Given a set of rectification candidates $\mathbf{r}_F$, how to construct a unified circuit (proof model) such that different combinations of rectification candidates can be tested for Equation (4) using SAT engines

For each rectification candidate r_i in $\mathbf{r}_F$, we define an auxiliary selection variable s_i to act as a switch to include r_i into the set of rectification signals. Combining Equation (6) and (7), we can construct a proof model $H(\mathbf{x}, s_i)$ as Figure 3.

Note that since each u_i is unate to F and is applicable to Equation (5), when modeling multi-fanout cases, we actually need not synthesize the function ' A ' nor represent $\forall r. F$ as the form in Equation (6). The proof model in Figure 3 acts as an implicit representation of Equation (6).

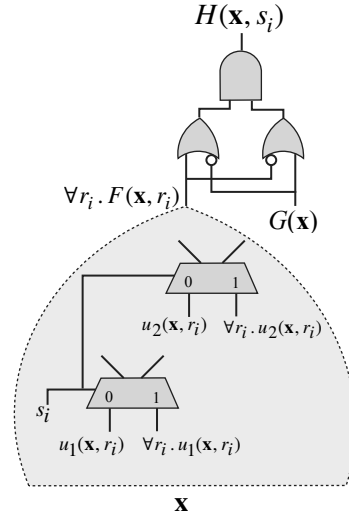


Fig. 3. Using selection variable s_i to construct the proof model

- How to extend Equation (7) for multiple POs

Up to this point, the modeling $H(\mathbf{x}, s_i)$ is considering single PO and is an exact model for checking the valid rectification signals.

For multiple PO cases, if we would like to find a set of rectification signals that can fix all the POs at one shot, we need to revise Equation (1) as:

$$\forall \mathbf{x}. \exists \mathbf{r}. (F_1(\mathbf{x}, \mathbf{r}) \equiv G_1(\mathbf{x})) \wedge (F_2(\mathbf{x}, \mathbf{r}) \equiv G_2(\mathbf{x})) \wedge \dots \quad (1')$$

Taking the inversion as in Equation (4), we have:

$$\exists \mathbf{x}. \forall \mathbf{r}. (F_1(\mathbf{x}, \mathbf{r}) \neq G_1(\mathbf{x})) \vee (F_2(\mathbf{x}, \mathbf{r}) \neq G_2(\mathbf{x})) \vee \dots \quad (4')$$

Let the proof model $H(\mathbf{x}, \mathbf{s}_F)$ for the multiple PO cases be the disjunction of the individual model $H(\mathbf{x}, s_i)$'s. That is:

$$H(\mathbf{x}, \mathbf{s}_F) = \bigvee_i H(\mathbf{x}, s_i) \quad (8)$$

As shown in Algorithm I, we use this proof model to check if the set of rectification candidates is a valid set of rectification signals. That is, if the proof " $\text{SAT}(H(\mathbf{x}, \mathbf{s}_F))$ " returns UNSATISFIABLE, we will save the corresponding set of signals $\mathbf{r}$ as a valid set of rectification signals.

However, since the universal quantifier $\forall r$ cannot be distributed over the disjunctive operator ' $\vee$ ', $H(\mathbf{x}, \mathbf{s}_F)$ is NOT an exact proof model for Equation (4').

C. Application of $H(\mathbf{x}, \mathbf{s}_F)$ in the ECO Algorithm

Since $H(\mathbf{x}, \mathbf{s}_F)$ is NOT an exact proof model for Equation (4'), we need to check whether using $H(\mathbf{x}, \mathbf{s}_F)$ in the ECO Algorithm is sound and complete.

Let's first look at these propositions.

Proposition 1: $H(\mathbf{x}, \mathbf{s}_F)$ is an under-approximated proof model of Equation (4')

Proof:

By definition,

$$H(\mathbf{x}, \mathbf{s}_F) = \bigvee_i H(\mathbf{x}, s_i) = \bigvee_i \forall \mathbf{r}. (F_i(\mathbf{x}, \mathbf{r}) \neq G_i(\mathbf{x})) \quad [1]$$

Since

$$\bigvee_i \forall \mathbf{r}. (F_i(\mathbf{x}, \mathbf{r}) \neq G_i(\mathbf{x})) \implies \forall \mathbf{r}. \bigvee_i (F_i(\mathbf{x}, \mathbf{r}) \neq G_i(\mathbf{x})) \quad [2]$$

Let's denote the proof model for Equation (4') as:

$$E(\mathbf{x}, \mathbf{s}_F) = \forall \mathbf{r}. \bigvee_i (F_i(\mathbf{x}, \mathbf{r}) \neq G_i(\mathbf{x})) \quad [3]$$

From [1], [2] and [3], we have --

$$H(\mathbf{x}, \mathbf{s}_F) \implies E(\mathbf{x}, \mathbf{s}_F)$$

We can conclude that $H(\mathbf{x}, \mathbf{s}_F)$ is an under-approximated proof model of Equation (4') [QED]

Proposition 2: If we use the approximated function $H(\mathbf{x}, \mathbf{s}_F)$ and identify a set of rectification signals $\mathbf{r}$, it is possible that we cannot generate a patch based on $\mathbf{r}$.

Proof:

If we use the approximated function $H(\mathbf{x}, \mathbf{s}_F)$ and identify a set of rectification signals $\mathbf{r}$, from Algorithm I, $\text{SAT}(H(\mathbf{x}, \mathbf{s}_F))$ must return UNSATISFIABILITY. That is, $H(\mathbf{x}, \mathbf{s}_F) = \text{false}$.

However, since $H(\mathbf{x}, \mathbf{s}_F) \implies E(\mathbf{x}, \mathbf{s}_F)$, it is possible that $E(\mathbf{x}, \mathbf{s}_F)$ is not 'false' and the proof $\text{SAT}(E(\mathbf{x}, \mathbf{s}_F))$ may return SATISFIABILITY. Therefore, it is possible that we cannot generate a patch. [QED]

Proposition 3: For a set of rectification candidates $\mathbf{r}_F$, if there exists a patch that can be generated from certain subset of $\mathbf{r}_F$, say $\mathbf{r}$, our approximated proof model $H(\mathbf{x}, \mathbf{s}_F)$ and thus our rectification signal validation algorithm (i.e. Algorithm I) will be able to guarantee such patch exists and identify this set of rectification signals $\mathbf{r}$.

Proof:

This can be easily derived from Proposition 2. Due to the space limit, we omit the proof here. [QED]

In conclusion, our proposed rectification signal validation algorithm can efficiently collect all the sets of valid rectification signals and thus obtain the optimal path. Although it is possible that we may suggest certain sets of rectification signals which are not valid for patch generation, experiments show that this rarely happens and thus the runtime overhead is very small.

IV. EXPERIMENTAL RESULTS

A. Experimental Settings

We present the experimental results of our proposed algorithm here. All experiments were conducted on a Linux workstation with 128GB RAM and a 2.2GHz Intel Xeon CPU. The testcases are from the 2017 CAD Contest [17].

To evaluate the quality of patches, we adopt the patch cost function from the 2021 CAD contest [18] as our criteria. We compare all our experimental results with the Conformal LEC tool by Cadence, using the Conformal ECO GXL 21.2 license, with the command *analyze eco* [19].

We leverage the open-source tool "runeco" [20] in Berkeley ABC [21] to generate the patches.

B. Comparison on Patch Costs and Runtime

Table I presents the experimental results on patch costs. The third column indicates the index for the occurrence of the optimal solution ("Opt Sol @") with respect to the number of sets of the rectification signals we have tested ("Sets tested"). It can be seen that with the rectification signal grouping and ranking heuristic and the simulation annealing technique, we can obtain the optimal patch with no more than 4 calls to the patch generation tool.

The 4th column reports the number of rectification signals for the optimal patch. For this ICCAD 2017 benchmark,

TABLE I. PATCH COST COMPARISON

Testcase	Design Size	Opt Sol @ / #Sets tested	#sigs for Opt Sol	Patch Cost			Patch Ratio	
				Baseline	Ours	Conformal ECO	Ours / Baseline	Ours / Conformal
test01	12	1/1	1	8	8	8	1	1
test02	1348	1/4	1	31	18	73	0.58	0.25
test03	2617	1/4	1	7	5	17	0.71	0.29
test04	95	1/3	1	3	3	3	1	1
test05	25092	4/4	3	180	159	191	0.88	0.83
test06	14064	-	-	-	-	367	-	-
test07	3180	4/4	3	86	63	132	0.73	0.48
test08	2759	2/3	4	108	108	86	1	1.26
test09	6353	3/4	20	132	123	157	0.93	0.78
test10	2216	-	-	-	-	353	-	-
test11	2278	-	-	-	-	95	-	-
test12	13804	3/4	1	1	1	1	1	1
test13	369	1/1	1	18	18	66	1	0.27
test14	1981	1/1	7	132	132	619	1	0.21
test15	1886	1/4	1	5	5	99	1	0.05
test16	2371	1/3	2	61	19	1076	0.31	0.02
test17	2910	2/3	1	150	145	196	0.96	0.74
test18	4860	1/4	2	19	16	245	0.84	0.07
test19	13349	-	-	-	-	169	-	-
test20	30816	1/1	6	59	59	87	1	0.68
Average				62.5	55.1	202	0.88	0.56

TABLE II. RUNTIME COMPARISON

Testcase	Our Runtime (sec)				Conformal Runtime (sec)	Our Runtime / Conformal Runtime
	Preparation Stage	Validation Stage	Patch Gen Stage	Total		
test01	0.01	0.01	1.24	1.26	31	0.04
test02	0.09	0.1	48.3	48.49	36.5	1.33
test03	0.12	0.09	8.2	8.41	31.4	0.27
test04	0.01	0.02	13.51	13.54	27.7	0.49
test05	1.11	0.12	415	416.23	51	8.36
test06	-	-	-	-	662	-
test07	2.62	22.12	600	624.74	57.8	10.81
test08	0.2	0.46	828	828.66	42	19.74
test09	1.01	1.23	145	147.24	36.5	4.03
test10	-	-	-	-	70.7	-
test11	-	-	-	-	64.7	-
test12	0.98	6.82	330	337.8	39.6	8.51
test13	0.02	0.03	1.87	1.92	32.9	0.06
test14	0.11	0.31	608	608.42	50.4	12.1
test15	0.12	0.46	36.5	37.08	42.4	0.88
test16	0.51	5.14	334	339.65	186	1.82
test17	1.01	4.78	155	160.79	37.7	4.29
test18	0.72	5.09	148	153.81	235	0.65
test19	-	-	-	-	1750	-
test20	0.88	1.22	253	255.1	49.8	5.13
Average	0.6	3.63	245.4	249.63	81.75	4.9

most cases can be fixed with less than four rectification signals.

We also implemented a baseline version in which the major difference is the heuristic grouping and ranking algorithm for the sets of the rectification signals during the patch generation stage. The results show that our full version has 12% advantages.

When compared with Conformal ECO, we can achieve 44% patch reductions on average. This asserts that with the exploration of more sets of rectification signals, we can consistently improve the patch size.

Nevertheless, we are unable to solve test06, 10, 11, and 19. The original *runeco* paper [20] specifically mentioned that these four cases require structural rewriting techniques to be resolved. Therefore, *runeco* is incapable of generating patches for them.

Table II is the runtime comparison. It is worth noting that 98% of our runtime is on the patch generation stage. This

implies that by transforming the problem from 2QBF to 1QBF, the runtime for collecting all the sets rectification signals is drastically reduced.

For the solved testcases, our runtime is about 4.9X when compared to Conformal ECO. On the one hand, this is due to the facts that we place more calls to patch generation (i.e. 3.44 times on average, from Table I). On the other hand, it says that there is room to improve the runtime for the open-source tool *runeco*.

C. Comparison and Analysis

To better evaluate the quality of the rectification signals and the optimality of the patch generation, we take the rectification signals collected by Conformal ECO and feed them to the *runeco* tool ((2) in Table III). When comparing it with our results (1), we can see that our ECO engine can optimize the patch for 20%. This indicates that our rectification signal collection and validation algorithm indeed contributes to the optimization of the patches.

On the other hand, if we compare (2) to (3) (i.e. to Conformal ECO alone), the *runeco* engine still outperforms Conformal ECO for 26%. It is not clear why the open-source tool performs better than commercial tool in these cases. However, the improvement (26%) is still less than the 44% improvement (ours vs. Conformal), indicating that our rectification signal collection and validation algorithm again plays an important role in optimizing the patches.

Ideally, we should have an experiment that collects our identified rectification signals and asks Conformal ECO to generate the patches. However, there is no such option in Conformal ECO and thus we cannot perform such comparison.

TABLE III. DIFFERENT RECTIFICATION SIGNAL + PATCH GEN

Testcase	Design Size	Patch Cost			(1) / (2)	(2) / (3)
		(1) Ours alone	(2) Conformal RS + runeco patch gen	(3) Conformal alone		
test01	12	8	8	8	1	1
test02	1348	18	39	73	0.46	0.53
test03	2617	5	5	17	1	0.29
test04	95	3	3	3	1	1
test07	3180	63	70	132	0.9	0.53
test09	6353	123	142	157	0.87	0.9
test12	13804	1	1	1	1	1
test17	2910	145	178	196	0.82	0.91
test18	4860	16	129	245	0.12	0.53
Average		36.9	54.7	182	0.80	0.74

V. CONCLUSION

We propose a novel rectification signal validation algorithm to efficiently collect all possible sets of rectification signals for optimal patch generation in functional ECO problems. Additionally, we introduce a grouping and ranking strategy for these rectification signal sets, allowing us to achieve optimal patch generation with minimal calls to the patch generation tool.

Experimental results indicate that by exploring multiple sets of rectification signals, our approach reduces patch costs by 44% on average compared to a commercial tool. However, some test cases involving structural changes fail when processed by the third-party patch generation tool (*runeco*). In the future, we plan to integrate our rectification signal validation technique with sweeping-based and fault-model-based ECO engines to handle a broader range of testcases.

REFERENCES

1. S.-L. Huang, W.-H. Lin, P.-K. Huang and C.-Y. Huang, "Match and Replace: A Functional ECO Engine for Multiterior Circuit Rectification," in IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 32, no. 3, pp. 467-478, March 2013, doi: 10.1109/TCAD.2012.2226456. <https://doi.org/10.1109/TCAD.2012.2226456>
2. K.-H. Chang, I. L. Markov and V. Bertacco, "Fixing Design Errors with Counterexamples and Resynthesis," 2007 Asia and South Pacific Design Automation Conference, Yokohama, Japan, 2007, pp. 944-949, doi: 10.1109/ASPDAC.2007.358111. Test in Europe Conference Exhibition (DATE), 1036-1041.
3. A. Veneris and I. Hajj, "Design error diagnosis and correction via test vector simulation," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 18, no. 12, pp. 1803-1816, Dec. 1999.
4. M. Abadir, J. Ferguson, and T. Kirkland, "Logic design verification via test generation," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 7, no. 1, pp. 138-148, Jan. 1988.
5. S. Krishnaswamy, H. Ren, N. Modi and R. Puri, "DeltaSyn: An efficient logic difference optimizer for ECO synthesis," 2009 IEEE/ACM International Conference on Computer-Aided Design - Digest of Technical Papers, San Jose, CA, USA, 2009, pp. 789-796, doi: 10.1145/1687399.1687546.
6. H. Ren, R. Puri, L. Reddy, S. Krishnaswamy, C. Washburn, J. Earl, and J. Keinert, "Intuitive ECO Synthesis for High Performance Circuits," in Proc. Design Autom. Test in Europe Conf. Exhib. (DATE), pp. 1002-1007, 2013.
7. D. Hoffmann and T. Kropf, "Efficient design error correction of digital circuits," in Proc. ICCD, Sep. 2000, pp. 465-472.
8. C.-C. Lin, K.-C. Chen, and M. Marek-Sadowska, "Logic synthesis for engineering change," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 18, no. 3, pp. 282-292, Mar. 1999.
9. B.-H. Wu, C.-J. Yang, C.-Y. Huang, and J.-H. Jiang, "A robust functional ECO engine by SAT proof minimization and interpolation techniques," in Proc. IEEE/ACM Int. Conf. ICCAD, Nov. 2010, pp. 729-734.
10. A. Ling, S. Brown, S. Safarpour, and J. Zhu, "Toward automated ECOs in FPGAs," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 30, no. 1, pp. 18-30, Jan. 2011.
11. K. -F. Tang, C. -A. Wu, P. -K. Huang and C. -Y. Huang, "Interpolation-based incremental ECO synthesis for multi-error logic rectification," 2011 48th ACM/EDAC/IEEE Design Automation Conference (DAC), San Diego, CA, USA, 2011, pp. 146-151. Design Automation Conference. 1-6.
12. K. -F. Tang, P. -K. Huang, C. -N. Chou and C. -Y. Huang, "Multi-patch generation for multi-error logic rectification by interpolation with cofactor reduction," 2012 Design, Automation & Test in Europe Conference & Exhibition (DATE), Dresden, Germany, 2012, pp. 1567-1572, doi: 10.1109/DATE.2012.6176722.
13. Celia Wrathall. Complete sets and the polynomial-time hierarchy. Theoretical Computer Science, 3(1):23-33, 1976.
14. A. M. Wahba, "BDD/AOG-based algorithm for multiple error rectification in combinational circuits," ICECS 2000. 7th IEEE International Conference on Electronics, Circuits and Systems (Cat. No.00EX445), Jounieh, Lebanon, 2000, pp. 658-661 vol.2, doi: 10.1109/ICECS.2000.912964.
15. Shi-Yu Huang, Kuang-Chien Chen and Kwang-Ting Cheng, "AutoFix: a hybrid tool for automatic logic rectification," in IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 18, no. 9, pp. 1376-1384, Sept. 1999, doi: 10.1109/43.784128.
16. A. Mishchenko, S. Chatterjee, R. Jiang, and R. K. Brayton, "FRAIGs: A unifying representation for logic synthesis and verification". ERL Technical Report, EECS Dept., UC Berkeley, March 2005..
17. 2017 CAD Contest, <https://www.iccad-contest.org/2017>.
18. 2021 CAD Contest, <https://www.iccad-contest.org/2021/>.
19. Conformal ECO Designer, https://www.cadence.com/en_US/home/tools/digital-design-and-signoff/functional-eco/conformal-eco-designer.html.
20. A. Q. Dao, N-Z Lee, L-C Chen, M. P-H Lin, J-H R. Jiang, A. Mishchenko, and R. Brayton, "Efficient Computation of ECO Patch Functions," 2018 55th ACM/ESDA/IEEE Design Automation Conference (DAC), San Francisco, CA, USA, 2018, pp. 1-6, doi: 10.1109/DAC.2018.8465898.
21. Berkeley ABC: A System for Sequential Synthesis and Verification, <https://people.eecs.berkeley.edu/~alanmi/abc/>.